

Testing practice (again) and OO philosophy

4/4/25

Administrivia

- Pre-course survey due tonight
- HW 2 (writing stub code and test cases for Car class) due Tuesday night

Recall: Automated testing with JUnit

- Want to be able to run/re-run tests of a method/class
- Create @Test methods that make calls and check return values with assertEquals: checking assertions about the code's behavior
- Optionally, set up a test fixture (1+ initialized objects) using an @Before method that is run before each test case

Recall: What assertions should we use?

- Look at description of the method/class

What promises does it make?

(promise = statement of what the method does on some input)

- Need assertions to
 - Check each promise (at least once)
 - Try edge cases when you barely hit a promise
 - Hit special cases: empty array, negative values, changing first/last in a list

Find promises and write assertions for VacuumCleaner

- @Before method to set up test fixture
- @Test methods w/ assertEquals(correct, call)
- To test that an exception is thrown:

```
try {  
    //do stuff that should generate exceptions  
    fail();  
} catch(ExceptionType ex) {  
    //NOTHING HERE  
}
```

Are you here?

- A. Absolutely!
- B. Yes
- C. I think so
- D. I was, but then I left
- E. Nope

From Friday: Abstract data types (ADTs)

- Set of operations supported on a particular kind of data
- Does not include implementation details
- Our example: List (ordered collection of items)
 - add, remove, get(i), size, etc

Implementation of List: ArrayList

- Array-based implementation of the List ADT

```
ArrayList<Point> list = new ArrayList<Point>();
```

```
Point p = new Point(x, y);
```

```
list.add(p);
```

```
for(int i=0; i<list.size(); i++) {  
    Point p = list.get(i);  
    //do something with p  
}
```


Interface: Java's way to express an ADT

- Essentially a list of method signatures

```
public interface List<T> {  
    public void add(T item);  
    public int size();  
    public T get(int index);  
    ...  
}
```

List of method signatures,
each followed by a semicolon

Using an interface

```
public class ArrayList<T> implements List<T> {  
    //must have methods for all promised signatures  
}
```

Using an interface

```
public class ArrayList<T> implements List<T> {  
    //must have methods for all promised signatures  
}
```

- Allows variable to store any implementing type:
List<String> list = new ArrayList<String>();

Using an interface

```
public class ArrayList<T> implements List<T> {  
    //must have methods for all promised signatures  
}
```

- Allows variable to store any implementing type:

```
List<String> list = new ArrayList<String>();
```

- Can define operation on any implementing type:

```
public String mostCommon(List<String> list) {  
    //find string with most occurrences for any List
```

This is what we did in lab

- PrintQueue interface defines some behavior
- WorkingPrintQueue (implements PrintQueue) does this correctly; all the methods work as advertised
- BadPrintQueue (implements PrintQueue) almost does this, but has a bug with some incorrect behavior
- You wrote test cases in terms of a PrintQueue p
 - When p referenced a WorkingPrintQueue, all the tests pass
 - When p referenced a BadPrintQueue, at least one failed

Object-oriented (OO) programming

- Program is bunch of interacting objects
- Classes (types of objects) specify what a kind of data an object stores (attributes) and what actions it can perform (methods)
- Objects are instances of a class with a specific value for each attribute

Design principles

- Encapsulation / information hiding: Hiding unnecessary details from other parts of the program
- Abstraction: Focus on what the class does instead of how (methods provided rather than their implementation)

Design principles

- Encapsulation / information hiding: Hiding unnecessary details from other parts of the program
- Abstraction: Focus on what the class does instead of how (methods provided rather than their implementation)

Upshot: Objects are responsible for themselves (keeping own data, performing own actions)

Design ideas

- Use cases: develop “stories” of how the code will be used
- Figure out the classes needed (nouns in stories)
- Draw relationships between classes in Unified Modeling Language (UML)

Application Programming Interface (API)

- Set of calls provided to the programmer by an implemented library
- Roughly an ADT, but typically comes with an implementation
- Example: Google Maps provides an API so that programmers can display their own data on a map